FINAL ASSIGNMENT REPORT

F1 DATABASE

Practical: Thursday 9-12

Nihalsharafuddeen
2117 4229

Introduction:

In this project, I have created a database to retrieve information regarding F1 2022 - 2024. This database has 5 entities: Teams, Championships, Circuits, Drivers, and Tickets. I have gathered all data for the entire 72 races that happened across 3 years. Then, I did ER modelling, relationship schema and data description to have a proper plan before starting with SQL. Then, the tables were implemented and inserted all the data to the relevant tables. Then I created few queries, stored procedure, triggers and views. In the end, I had also used Python to connect to the database and do some tasks.

Why have I selected the Entities, Relationships, and Data Types?

The database was made to get information about the F1 Races from 2022-2024 and about the ticketing information. I have chosen these based on the resources I found, including the official F1 Website. I have added all the main information including drivers' and teams' details, race results and standings and the ticketing information. These are the things that people usually look for in the F1 database.

Entity Sets:

Entity Set	**Key**	**Attributes**
Teams	Team_Name	Base
Championships	Year	Total_Races
Circuits	Circuit_ID	Track , Circuit_Name, Location
Drivers	Driver_ID	Driver_Number, Driver_Name, Team_Name, DOB, Nationality
Tickets	Ticket_ID	Type

Relationship Tables:

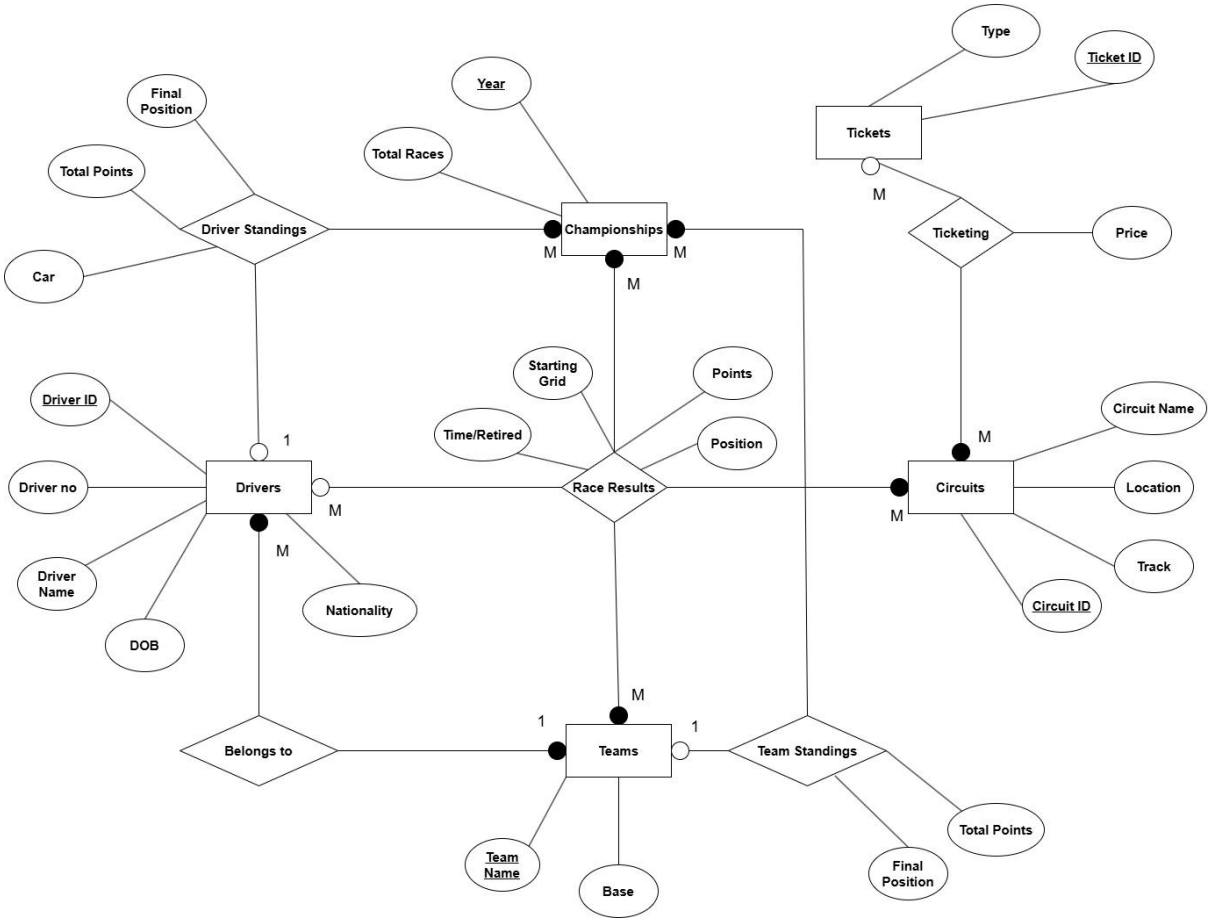
Relationship Set	**Between Entity Sets**	**Attributes**
Belongs_To	Drivers, Teams	
Driver_Standings	Driver_Standings, Championships	Final_Position, Total_Points
Team_Standings	Team_Standings, Championships	Final_Position, Total_Points
Ticketing	Tickets, Circuits	Price
Race_Results	Drivers, Championships, Circuits	Position, Points, Time

Constraints Table (Cardinality):

Relationship Set	Cardinality	Description
Belongs_To	M : 1	A driver belongs to one team, but a team has many drivers.
Driver_Standings	1 : M	A championship has multiple driver standings, but each driver standing belongs to one championship.
Team_Standings	1 : M	A championship has multiple team standings, but each team standing belongs to one championship.
Ticketing	M : M	A ticket type applies to multiple races, and each race has different ticket types.
Race_Results	M : M : M	A race happens in a championship and at a circuit; multiple races occur in a championship with multiple drivers.

Constraints Table (Participation):

Relationship Set	Participation	Description
Belongs_To	Drivers - total, Teams - total	A driver must belong to a team, and a team can't exist without drivers.
Driver_Standings	Drivers - partial, Championships - total	A driver can exist without participating in a championship, but each championship must have drivers.
Team_Standings	Teams - partial, Championships - total	A team can exist without participating in a championship, but each championship must have teams.
Ticketing	Tickets - partial, Championships - total, Circuits - total	A ticket type may not be used in all championships or circuits, but ticketing must involve tickets.
Race_Results	Drivers - partial, Championships - total, Circuits - total	A race result must belong to a championship and a circuit, and every championship must have races.


Type
Ticket ID
Tickets
Year
Final Position
Total Races
Total Points
Driver Standings
Championships
Car
M
M
M
M
Ticketing
Price
Starting Grid
Points
Driver ID
Time/Retired
Position
Circuit Name
1
M
Driver no
Drivers
Race Results
Circuits
Location
M
M
Driver Name
Track
Nationality
Circuit ID
DOB
M
1
1
Belongs to
Teams
Team Standings
Total Points
Team Name
Final Position
Base

Relational Schema:

Teams (Team_Name, Base)

Championships (Year, Total_Races)

Circuits (Circuit_ID, Circuit_Name, Track, Location)

Tickets (Ticket_ID, Type)

Drivers (Driver_ID, Driver_Number, Driver_Name, Team_Name, DOB, Nationality)
FK (Team_Name) REF Teams(Team_Name) ON DELETE RESTRICT ON UPDATE CASCADE

Driver_Standings (Year, Final_Position, Driver_ID, Car, Total_Points)
FK (Driver_ID) REF Drivers(Driver_ID) ON DELETE RESTRICT ON UPDATE CASCADE
FK (Year) REF Championships(Year) ON DELETE RESTRICT ON UPDATE CASCADE

Team_Standings (Year, Final_Position, Team_Name, Total_Points)
FK (Team_Name) REF Teams(Team_Name) ON DELETE RESTRICT ON UPDATE CASCADE
FK (Year) REF Championships(Year) ON DELETE RESTRICT ON UPDATE CASCADE

Ticketing (Ticket_ID, Circuit_ID, Price)
FK (Ticket_ID) REF Tickets(Ticket_ID) ON DELETE RESTRICT ON UPDATE CASCADE
FK (Circuit_ID) REF Circuits(Circuit_ID) ON DELETE RESTRICT ON UPDATE CASCADE

Race_Results (Year, Circuit_ID, Driver_ID, Team_Name, Position, Starting_Grid, Time_or_Retired, Points)

FK (Year) REF Championships(Year) ON DELETE RESTRICT ON UPDATE CASCADE

FK (Circuit_ID) REF Circuits(Circuit_ID) ON DELETE RESTRICT ON UPDATE CASCADE

FK (Driver_ID) REF Drivers(Driver_ID) ON DELETE RESTRICT ON UPDATE CASCADE

FK (Team_Name) REF Teams(Team_Name) ON DELETE RESTRICT ON UPDATE CASCADE

Data Description:

Table: Teams

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Team_Name	VARCHAR	100	N	Y	Team Name	-
Base	VARCHAR	100	N	-	Team Base	-

Table: Championships

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Year	INT	-	N	Y	Championship Year	-
Total_Races	INT	-	N	-	Total Races Held	-

Table: Circuits

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Circuit_ID	CHAR	3	N	Y	Circuit Identifier	-
Circuit_Name	VARCHAR	50	N	-	Name of Circuit	-
Track	VARCHAR	20	N	-	Track Identifier	-
Location	VARCHAR	20	N	-	Location of Circuit	-

Table: Tickets

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Ticket_ID	INT	-	N	Y	Ticket ID	-
Type	VARCHAR	30	N	-	Ticket Type	-

Table: Drivers

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Driver_ID	CHAR	5	N	Y	Unique Driver ID	-
Driver_Number	INT	-	N	-	Driver's Number	-
Driver_Name	VARCHAR	100	N	-	Driver's Name	-
Team_Name	VARCHAR	100	N	-	Team Associated	FK REFERENCES Teams(Team_Name) ON DELETE RESTRICT ON UPDATE CASCADE
DOB	DATE	-	N	-	Date of Birth	-
Nationality	VARCHAR	50	N	-	Nationality	-

Table: Driver_Standings

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Year	INT	-	N	Y	Championship Year	FK REFERENCES Championships(Year) ON DELETE CASCADE ON UPDATE CASCADE
Final_Position	INT	-	N	Y	Final Position in Standings	-
Driver_ID	CHAR	5	N	-	Unique Driver Identifier	FK REFERENCES Drivers(Driver_ID) ON DELETE

						RESTRICT ON UPDATE CASCADE
Car	VARCHAR	50	N	-	Car Used	-
Total_Points	INT	-	N	-	Total Points Earned	-

Table: Team_Standings

Attribute	**Type**	**Size**	**Null**	**Primary Key**	**Description**	**Other Constraints**
Year	INT	-	N	Y	Championship Year	FK REFERENCES Championships(Year) ON DELETE CASCADE ON UPDATE CASCADE
Final_Position	INT	-	N	Y	Final Position in Standings	-
Team_Name	VARCHAR	100	N	-	Team Name	FK REFERENCES Teams(Team_Name) ON DELETE CASCADE ON UPDATE CASCADE
Total_Points	INT	-	N	-	Total Points Earned	-

Table: Race_Results

Attribute	Type	Size	Null	Primary Key	Description	Other Constraints
Year	INT	-	N	Y	Championship Year	FK REFERENCES Championships(Year) ON DELETE CASCADE ON UPDATE CASCADE
Circuit_ID	CHAR	3	N	Y	Circuit Identifier	FK REFERENCES Circuits(Circuit_ID) ON DELETE CASCADE ON UPDATE CASCADE
Driver_ID	CHAR	5	N	Y	Unique Driver Identifier	FK REFERENCES Drivers(Driver_ID) ON DELETE CASCADE ON UPDATE CASCADE
Team_Name	VARCHAR	100	N	-	Team Name	FK REFERENCES Teams(Team_Name) ON DELETE CASCADE ON UPDATE CASCADE
Position	INT	-	N	-	Final Position in Race	-
Starting_Grid	INT	-	N	-	Starting Grid Position	-
Time_or_Retired	VARCHAR	50	N	-	Race Completion Status	-
Points	INT	-	N	-	Points Earned	-

Assumption made in the database:

I have assumed that the team name will be always unique, because I could not find any duplicate team name in the past seasons. I have added IDs for drivers, because there were no unique attributes in their details. I have also done the same for circuits as their names are long and 2 circuits might be in the same location.

Business Rules:

No.	Description
1.	A driver can only be associated with one team in a given season.
2.	Each race result for a driver can only have one finishing position per race.
3.	A race is tied to one specific circuit for its location, and the circuit can only host one race in a given year.
4.	Every team must have at least one driver participating in a season.
5.	Each driver is identified by a unique number during the championship season.
6.	Teams may field two drivers each season but can switch drivers between seasons.
7.	Teams must enter a race to earn points for the Constructors' Championship.
8.	Only drivers who actually raced and finished (or retired) in a race will have their results recorded.
9.	Each championship season must have at least one race held for standings to be calculated.
10.	A driver cannot participate in multiple seasons at once but can change teams in between seasons.

Implementing the database:

1. Creating Tables

Firstly, I created a Database called F1_21174229. Then, I created all the tables following the relational schema and data description.

```sql
CREATE TABLE Teams (
    Team_Name VARCHAR(100) PRIMARY KEY,
    Base VARCHAR(30) NOT NULL
);

CREATE TABLE Championships (
    Year INT PRIMARY KEY,
    Total_Races INT NOT NULL
);

CREATE TABLE Circuits (
    Circuit_ID CHAR(3) PRIMARY KEY,
    Circuit_Name VARCHAR(50),
    Track VARCHAR(20),
    Location VARCHAR(20)
);

CREATE TABLE Tickets (
    Ticket_ID INT PRIMARY KEY,
    Type VARCHAR(30)
);

CREATE TABLE Drivers (
    Driver_ID CHAR(5) PRIMARY KEY,
    Driver_Number INT NOT NULL,
    Driver_Name VARCHAR(100) NOT NULL,
    Team_Name VARCHAR(100) NOT NULL,
    DOB DATE NOT NULL,
    Nationality VARCHAR(50) NOT NULL,
    FOREIGN KEY (Team_Name) REFERENCES Teams (Team_Name) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

```sql
CREATE TABLE Driver_Standings (
    Year INT NOT NULL,
    Final_Position INT NOT NULL,
    Driver_ID CHAR(5),
    Car VARCHAR(30) NOT NULL,
    Total_Points INT NOT NULL,
    PRIMARY KEY (Year, Driver_ID),
    FOREIGN KEY (Driver_ID) REFERENCES Drivers (Driver_ID) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (Year) REFERENCES Championships (Year) ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE Team_Standings (
    Year INT NOT NULL,
    Final_Position INT NOT NULL,
    Team_Name VARCHAR(100) NOT NULL,
    Total_Points INT NOT NULL,
    PRIMARY KEY (Year, Team_Name),
    FOREIGN KEY (Team_Name) REFERENCES Teams (Team_Name) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (Year) REFERENCES Championships (Year) ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE Ticketing (
    Ticket_ID INT NOT NULL,
    Circuit_ID CHAR(3) NOT NULL,
    Price DECIMAL(8,2) NOT NULL,
    PRIMARY KEY (Ticket_ID, Circuit_ID),
    FOREIGN KEY (Ticket_ID) REFERENCES Tickets (Ticket_ID) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (Circuit_ID) REFERENCES Circuits (Circuit_ID) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

```sql
CREATE TABLE Race_Results (
    Year INT NOT NULL,
    Circuit_ID CHAR(3) NOT NULL,
    Position INT,
    Driver_ID CHAR(5),
    Team_Name VARCHAR(100) NOT NULL,
    Starting_Grid INT NOT NULL,
    Time_or_Retired VARCHAR(10) NOT NULL,
    Points INT NOT NULL,
    PRIMARY KEY (Year, Circuit_ID, Driver_ID),
    FOREIGN KEY (Year) REFERENCES Championships (Year) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (Circuit_ID) REFERENCES Circuits (Circuit_ID) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (Driver_ID) REFERENCES Drivers (Driver_ID) ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (Team_Name) REFERENCES Teams (Team_Name) ON DELETE RESTRICT ON UPDATE CASCADE
);
```

```
mysql> SOURCE Tables.sql;
Query OK, 0 rows affected (0.13 sec)

Query OK, 0 rows affected (0.10 sec)

Query OK, 0 rows affected (0.10 sec)

Query OK, 0 rows affected (0.16 sec)

Query OK, 0 rows affected (0.14 sec)

Query OK, 0 rows affected (0.15 sec)

Query OK, 0 rows affected (0.11 sec)

Query OK, 0 rows affected (0.11 sec)

Query OK, 0 rows affected (0.10 sec)

Query OK, 0 rows affected (0.13 sec)

Query OK, 0 rows affected (0.13 sec)
```

2. Inserting Data

Then, I went through different F1 related databases to retrieve the information. I have referred to multiple sources including the official F1 website, the databases that were provided with the assignment and some other databases online. Then, I have used the Insert statement to add the data to the table. I also made all the requirements were met while adding the data.

```
INSERT INTO Teams (Team_Name, Base) VALUES
    ('Red Bull Racing', 'Milton Keynes, United Kingdom'),
    ('Ferrari', 'Maranello, Italy'),
    ('Mercedes', 'Brackley, United Kingdom'),
    ('Alpine', 'Enstone, United Kingdom'),
    ('McLaren', 'Woking, United Kingdom'),
    ('Alfa Romeo', 'Hinwil, Switzerland'),
    ('Aston Martin', 'Silverstone, United Kingdom'),
    ('Haas', 'Kannapolis, United States'),
    ('AlphaTauri', 'Faenza, Italy'),
    ('Williams', 'Grove, United Kingdom'),
    ('RB', 'Faenza, Italy'),
    ('Kick Sauber', 'Hinwil, Switzerland');

INSERT INTO Championships(Year, Total_Races) VALUES
    (2022, 22),
    (2023, 22),
    (2024, 24);

INSERT INTO Drivers (Driver_ID, Driver_name, Driver_number, Team_Name, Nationality, DOB) VALUES
    ('F1001', 'Max Verstappen', 1, 'Red Bull Racing', 'Netherlands', '1997-09-30'),
    ('F1002', 'Charles Leclerc', 16, 'Ferrari', 'Monaco', '1997-10-16'),
    ('F1003', 'Sergio Perez', 11, 'Red Bull Racing', 'Mexico', '1990-01-26'),
    ('F1004', 'George Russell', 63, 'Mercedes', 'United Kingdom', '1998-02-15'),
    ('F1005', 'Carlos Sainz', 55, 'Ferrari', 'Spain', '1994-09-01'),
    ('F1006', 'Lewis Hamilton', 44, 'Mercedes', 'United Kingdom', '1985-01-07'),
    ('F1007', 'Lando Norris', 4, 'McLaren', 'United Kingdom', '1999-11-13'),
    ('F1008', 'Esteban Ocon', 31, 'Alpine', 'France', '1996-09-17'),
    ('F1009', 'Fernando Alonso', 14, 'Aston Martin', 'Spain', '1981-07-29'),
    ('F1010', 'Valtteri Bottas', 77, 'Kick Sauber', 'Finland', '1989-08-28'),
    ('F1011', 'Daniel Ricciardo', 3, 'RB', 'Australia', '1989-07-01'),
```

```
INSERT INTO Circuits (Circuit_ID, Circuit_Name, Track, Location) VALUES
('C01', 'Bahrain International Circuit', 'Bahrain', 'Sakhir'),
('C02', 'Jeddah Corniche Circuit', 'Saudi Arabia', 'Jeddah'),
('C03', 'Albert Park Circuit', 'Australia', 'Melbourne'),
('C04', 'Suzuka International Racing Course', 'Japan', 'Suzuka'),
('C05', 'Shanghai International Circuit', 'China', 'Shanghai'),
('C06', 'Miami International Autodrome', 'Miami', 'Miami'),
('C07', 'Imola Circuit (Autodromo Enzo e Dino Ferrari)', 'Emilia Romagna', 'Imola'),
('C08', 'Circuit de Monaco', 'Monaco', 'Monte Carlo'),
('C09', 'Circuit Gilles Villeneuve', 'Canada', 'Montreal'),
('C10', 'Circuit de Barcelona-Catalunya', 'Spain', 'Barcelona'),
('C11', 'Red Bull Ring', 'Austria', 'Spielberg'),
('C12', 'Silverstone Circuit', 'Great Britain', 'Silverstone'),
('C13', 'Hungaroring', 'Hungary', 'Mogyoród'),
('C14', 'Circuit de Spa-Francorchamps', 'Belgium', 'Stavelot'),
('C15', 'Circuit Paul Ricard', 'France', 'Le Castellet'),
('C16', 'Circuit Zandvoort', 'Netherlands', 'Zandvoort'),
('C17', 'Monza Circuit (Autodromo Nazionale Monza)', 'Italy', 'Monza'),
('C18', 'Baku City Circuit', 'Azerbaijan', 'Baku'),
('C19', 'Marina Bay Street Circuit', 'Singapore', 'Singapore'),
('C20', 'Circuit of the Americas', 'United States', 'Austin'),
('C21', 'Autódromo Hermanos Rodríguez', 'Mexico', 'Mexico City'),
('C22', 'Interlagos (Autódromo José Carlos Pace)', 'Brazil', 'São Paulo'),
('C23', 'Las Vegas Strip Circuit', 'Las Vegas', 'Las Vegas'),
('C24', 'Lusail International Circuit', 'Qatar', 'Lusail'),
```

```sql
INSERT INTO Tickets (Ticket_ID, Type) VALUES
(1, 'General Admission (GA)'),
(2, 'Grandstand Tickets'),
(3, 'Main Grandstand'),
(4, 'Turn Grandstands'),
(5, 'Team Hospitality'),
(6, 'Camping Tickets'),
(7, 'Premium Grandstand Tickets'),
(8, 'Paddock Club');
```

```sql
/*Driver Standings 2024*/
INSERT INTO Driver_Standings (Year, Final_Position, Driver_ID, Car, Total_Points) VALUES
(2024, 1, 'F1001', 'Red Bull Racing Honda RBPT', 437),
(2024, 2, 'F1007', 'McLaren Mercedes', 374),
(2024, 3, 'F1002', 'Ferrari', 356),
(2024, 4, 'F1024', 'McLaren Mercedes', 292),
(2024, 5, 'F1005', 'Ferrari', 290),
(2024, 6, 'F1004', 'Mercedes', 245),
(2024, 7, 'F1006', 'Mercedes', 223),
(2024, 8, 'F1003', 'Red Bull Racing Honda RBPT', 152),
(2024, 9, 'F1009', 'Aston Martin Aramco Mercedes', 70),
(2024, 10, 'F1014', 'Alpine Renault', 42),
(2024, 11, 'F1023', 'Haas Ferrari', 41),
(2024, 12, 'F1017', 'RB Honda RBPT', 30),
(2024, 13, 'F1015', 'Aston Martin Aramco Mercedes', 24),
(2024, 14, 'F1008', 'Alpine Renault', 23),
(2024, 15, 'F1013', 'Haas Ferrari', 16),
(2024, 16, 'F1019', 'Williams Mercedes', 12),
(2024, 17, 'F1011', 'RB Honda RBPT', 12),
```

```sql
INSERT INTO Team_Standings (Year, Final_Position, Team_Name, Total_Points) VALUES
(2024, 1, 'McLaren', 666),
(2024, 2, 'Ferrari', 652),
(2024, 3, 'Red Bull Racing', 589),
(2024, 4, 'Mercedes', 468),
(2024, 5, 'Aston Martin', 94),
(2024, 6, 'Alpine', 65),
(2024, 7, 'Haas', 58),
(2024, 8, 'RB', 46),
(2024, 9, 'Williams', 17),
(2024, 10, 'Kick Sauber', 4);
```

```sql
INSERT INTO Ticketing (Ticket_ID, Circuit_ID, Price) VALUES
/*Bahrain*/
(1, 'C01', 150.00),
(2, 'C01', 300.00),
(3, 'C01', 500.00),
(4, 'C01', 400.00),
(5, 'C01', 1200.00),
(6, 'C01', 100.00),
(7, 'C01', 600.00),
(8, 'C01', 5000.00),
```

```
INSERT INTO Race_Results (Circuit_ID, Position, Driver_ID, Team_Name, Starting_Grid, Time_or_Retired, Points, Year) VALUES
('C01', 1, 'F1002', 'Ferrari', 1, '37:33.6', 26, 2022),
('C01', 2, 'F1005', 'Ferrari', 3, '5.598', 18, 2022),
('C01', 3, 'F1006', 'Mercedes', 5, '9.675', 15, 2022),
('C01', 4, 'F1004', 'Mercedes', 9, '11.211', 12, 2022),
('C01', 5, 'F1013', 'Haas', 7, '14.754', 10, 2022),
('C01', 6, 'F1010', 'Alfa Romeo', 6, '16.119', 8, 2022),
('C01', 7, 'F1008', 'Alpine', 11, '19.423', 6, 2022),
('C01', 8, 'F1017', 'AlphaTauri', 16, '20.386', 4, 2022),
('C01', 9, 'F1009', 'Alpine', 8, '22.39', 2, 2022),
('C01', 10, 'F1018', 'Alfa Romeo', 15, '23.064', 1, 2022),
('C01', 11, 'F1016', 'Haas', 12, '32.574', 0, 2022),
('C01', 12, 'F1015', 'Aston Martin', 19, '45.873', 0, 2022),
('C01', 13, 'F1019', 'Williams', 14, '53.932', 0, 2022),
('C01', 14, 'F1011', 'Mclaren', 18, '54.975', 0, 2022),
('C01', 15, 'F1007', 'Mclaren', 13, '56.335', 0, 2022),
('C01', 16, 'F1021', 'Williams', 20, '61.795', 0, 2022),
('C01', 17, 'F1023', 'Aston Martin', 17, '63.829', 0, 2022),
('C01', 18, 'F1003', 'Red Bull Racing', 4, 'DNF', 0, 2022),
('C01', 19, 'F1001', 'Red Bull Racing', 2, 'DNF', 0, 2022),
('C01', NULL, 'F1014', 'AlphaTauri', 10, 'DNF', 0, 2022),
('C02', 1, 'F1001', 'Red Bull Racing', 4, '24:19.3', 25, 2022),
('C02', 2, 'F1002', 'Ferrari', 2, '0.549', 19, 2022),
('C02', 3, 'F1005', 'Ferrari', 3, '8.097', 15, 2022),
('C02', 4, 'F1003', 'Red Bull Racing', 1, '10.8', 12, 2022),
('C02', 5, 'F1004', 'Mercedes', 6, '32.732', 10, 2022),
```

```
mysql> SOURCE Values.sql;
Query OK, 12 rows affected (0.05 sec)
Records: 12  Duplicates: 0  Warnings: 0

Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

Query OK, 30 rows affected (0.03 sec)
Records: 30  Duplicates: 0  Warnings: 0

Query OK, 25 rows affected (0.04 sec)
Records: 25  Duplicates: 0  Warnings: 0

Query OK, 8 rows affected (0.03 sec)
Records: 8  Duplicates: 0  Warnings: 0
```

```
mysql> SOURCE Race_Results.sql;
Query OK, 440 rows affected (0.10 sec)
Records: 440  Duplicates: 0  Warnings: 0

Query OK, 440 rows affected (0.19 sec)
Records: 440  Duplicates: 0  Warnings: 0

Query OK, 479 rows affected (0.11 sec)
Records: 479  Duplicates: 0  Warnings: 0
```

Use of the database

Queries:

1. Displays all the circuit names that start with the letter B. This can be helpful when you need to filter down the circuit details based on a common pattern, like starting of the name.

```
/* 1. Select all circuits which has the name of the track start with B */
SELECT Track, Location
FROM Circuits
WHERE Track LIKE 'B%';
```

```
mysql> SOURCE Queries.sql;
+---------+------------+
| Track   | Location   |
+---------+------------+
| Bahrain | Sakhir     |
| Belgium | Stavelot   |
| Brazil  | São Paulo  |
+---------+------------+
3 rows in set (0.00 sec)
```

2. Select all races that were held in 2023. This is a very useful query to understand the circuits that are hosting the races in a specific year, like 2023.

```
/* 2. Select all races that were held in 2023 */
SELECT DISTINCT Year, Circuit_ID
FROM Race_Results
WHERE Year = 2023;
```

```
+------+------------+
| Year | Circuit_ID |
+------+------------+
| 2023 | C01        |
| 2023 | C02        |
| 2023 | C03        |
| 2023 | C04        |
| 2023 | C06        |
| 2023 | C08        |
| 2023 | C09        |
| 2023 | C10        |
| 2023 | C11        |
| 2023 | C12        |
| 2023 | C13        |
| 2023 | C14        |
| 2023 | C16        |
| 2023 | C17        |
| 2023 | C18        |
| 2023 | C19        |
| 2023 | C20        |
| 2023 | C21        |
| 2023 | C22        |
| 2023 | C23        |
| 2023 | C24        |
| 2023 | C25        |
+------+------------+
22 rows in set (0.00 sec)
```

3. Select the top 3 drivers for every year. This is also very important as users usually look for the drivers that were on the podium every year.

```sql
/* 3. Select the top 3 drivers for every year */
SELECT ds.Year, ds.Final_Position, ds.Driver_ID, d.Driver_Name
FROM Driver_Standings ds
JOIN Drivers d ON ds.Driver_ID = d.Driver_ID
WHERE ds.Final_Position IN (1, 2, 3)
ORDER BY ds.Year, ds.Final_Position;
```

```
+------+----------------+-----------+-----------------+
| Year | Final_Position | Driver_ID | Driver_Name     |
+------+----------------+-----------+-----------------+
| 2022 |              1 | F1001     | Max Verstappen  |
| 2022 |              2 | F1002     | Charles Leclerc |
| 2022 |              3 | F1003     | Sergio Perez    |
| 2023 |              1 | F1001     | Max Verstappen  |
| 2023 |              2 | F1003     | Sergio Perez    |
| 2023 |              3 | F1006     | Lewis Hamilton  |
| 2024 |              1 | F1001     | Max Verstappen  |
| 2024 |              2 | F1007     | Lando Norris    |
| 2024 |              3 | F1002     | Charles Leclerc |
+------+----------------+-----------+-----------------+
9 rows in set (0.00 sec)
```

4. Select all drivers that are younger than 30 years old. This is very useful to understand the young drivers who will probably continue in the F1 in the upcoming years.

```sql
/* 4. Select all drivers that are younger than 30 years old */
SELECT Driver_ID, Driver_Name, DOB, TIMESTAMPDIFF(YEAR, DOB, CURDATE()) AS Age
FROM Drivers
WHERE TIMESTAMPDIFF(YEAR, DOB, CURDATE()) < 30;
```

```
+-----------+-----------------+------------+------+
| Driver_ID | Driver_Name     | DOB        | Age  |
+-----------+-----------------+------------+------+
| F1001     | Max Verstappen  | 1997-09-30 |   27 |
| F1002     | Charles Leclerc | 1997-10-16 |   27 |
| F1004     | George Russell  | 1998-02-15 |   27 |
| F1007     | Lando Norris    | 1999-11-13 |   25 |
| F1008     | Esteban Ocon    | 1996-09-17 |   28 |
| F1014     | Pierre Gasly    | 1996-02-07 |   29 |
| F1015     | Lance Stroll    | 1998-10-29 |   26 |
| F1016     | Mick Schumacher | 1999-03-22 |   26 |
| F1017     | Yuki Tsunoda    | 2000-05-11 |   24 |
| F1018     | Guanyu Zhou     | 1999-05-30 |   25 |
| F1019     | Alexander Albon | 1996-03-23 |   29 |
| F1021     | Nicholas Latifi | 1995-06-29 |   29 |
| F1024     | Oscar Piastri   | 2001-04-06 |   23 |
| F1025     | Liam Lawson     | 2002-02-11 |   23 |
| F1026     | Logan Sargeant  | 2000-12-31 |   24 |
| F1027     | Oliver Bearman  | 2005-05-08 |   19 |
| F1028     | Oliver Bearman  | 2005-05-08 |   19 |
| F1029     | Franco Colapinto | 2003-05-27 |   21 |
| F1030     | Jack Doohan     | 2003-01-20 |   22 |
+-----------+-----------------+------------+------+
19 rows in set (0.00 sec)
```

5. Select all drivers that were disqualified. This is very important and useful to understand how often disqualifications happen and how it affects their final position in the standings.

```sql
/* 5. Select all drivers that were disqualified */
SELECT Year, Driver_ID
FROM Race_Results
WHERE Time_or_Retired = 'DSQ';
```

```
+------+-----------+
| Year | Driver_ID |
+------+-----------+
| 2024 | F1023     |
+------+-----------+
1 row in set (0.01 sec)
```

6. Count the number of times each driver has won a race. This is useful to understand how many different winners we get in the span of 2 or 3 years. Also to understand who all have dominated in the recent championships.

```sql
/* 6. Count the number of times each driver has won a race */
SELECT r.Driver_ID, d.Driver_Name, COUNT(*) AS Wins
FROM Race_Results r
JOIN Drivers d ON r.Driver_ID = d.Driver_ID
WHERE r.Position = 1
GROUP BY r.Driver_ID, d.Driver_Name
ORDER BY Wins DESC;
```

```
+-----------+-----------------+------+
| Driver_ID | Driver_Name     | Wins |
+-----------+-----------------+------+
| F1001     | Max Verstappen  |   43 |
| F1002     | Charles Leclerc |    6 |
| F1003     | Sergio Perez    |    4 |
| F1005     | Carlos Sainz    |    4 |
| F1007     | Lando Norris    |    4 |
| F1004     | George Russell  |    3 |
| F1006     | Lewis Hamilton  |    2 |
| F1024     | Oscar Piastri   |    2 |
+-----------+-----------------+------+
8 rows in set (0.00 sec)
```

7. Select all drivers who have raced for a team that won the championship from 2022-2024. To know who all were part of the winning team.

```sql
/* 7. Select all drivers who have raced for a team that won the championship from 2022-2024 */
SELECT DISTINCT d.Driver_Name
FROM Race_Results rr
JOIN Drivers d ON rr.Driver_ID = d.Driver_ID
JOIN Team_Standings ts ON rr.Team_Name = ts.Team_Name
WHERE ts.Final_Position = 1
AND ts.Year BETWEEN 2022 AND 2024;
```

```
+------------------+
| Driver_Name      |
+------------------+
| Max Verstappen   |
| Sergio Perez     |
| Lando Norris     |
| Daniel Ricciardo |
| Oscar Piastri    |
+------------------+
5 rows in set (0.00 sec)
```

8. Select all teams and the drivers they have had. To know the total number of drivers that were part of a team in the past 3 years, and their names.

```sql
/* 8. Select all teams and the drivers they have had */
SELECT t.Team_Name, COUNT(d.Driver_Name) AS NumberOfDrivers,
        GROUP_CONCAT(SUBSTRING_INDEX(d.Driver_Name, ' ', 1) SEPARATOR ', ') AS Driver
FROM Teams t
JOIN Drivers d ON t.Team_Name = d.Team_Name
GROUP BY t.Team_Name
ORDER BY t.Team_Name;
```

```
+-----------------+-----------------+-----------------------------------------+
| Team_Name       | NumberOfDrivers | Driver                                  |
+-----------------+-----------------+-----------------------------------------+
| AlphaTauri      |               1 | Nyck                                    |
| Alpine          |               3 | Esteban, Pierre, Jack                   |
| Aston Martin    |               3 | Fernando, Sebastian, Lance              |
| Ferrari         |               3 | Charles, Carlos, Oliver                 |
| Haas            |               4 | Kevin, Mick, Nico, Oliver               |
| Kick Sauber     |               2 | Valtteri, Guanyu                        |
| McLaren         |               2 | Lando, Oscar                            |
| Mercedes        |               2 | George, Lewis                           |
| RB              |               3 | Daniel, Yuki, Liam                      |
| Red Bull Racing |               2 | Max, Sergio                             |
| Williams        |               5 | Alexander, Nyck, Nicholas, Logan, Franco |
+-----------------+-----------------+-----------------------------------------+
11 rows in set (0.00 sec)
```

Advanced Concepts:

Stored Procedures:

1. Add New Driver

```
/* Procedure to insert driver details if the team exists  */
DELIMITER //
DROP PROCEDURE IF EXISTS Add_Driver;

CREATE PROCEDURE Add_Driver (
    IN p_Driver_ID CHAR(5),
    IN p_Driver_Number INT,
    IN p_Driver_Name VARCHAR(100),
    IN p_Team_Name VARCHAR(100),
    IN p_DOB DATE,
    IN p_Nationality VARCHAR(50)
)
BEGIN
    DECLARE team_exists INT;

    SELECT COUNT(*) INTO team_exists FROM Teams WHERE Team_Name = p_Team_Name;

    IF team_exists = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Provided Team_Name does not exist in Teams table';
    ELSE
        INSERT INTO Drivers (Driver_ID, Driver_Number, Driver_Name, Team_Name, DOB, Nationality)
        VALUES (p_Driver_ID, p_Driver_Number, p_Driver_Name, p_Team_Name, p_DOB, p_Nationality);
    END IF;
END //

DELIMITER ;
```

```
mysql> CALL Add_Driver('F1031', 33, 'Max Verstappen', 'Red Bull Racing', '1997-0
9-30', 'Dutch');
Query OK, 1 row affected (0.02 sec)
```

2. Get the total number of races each driver participated in

```
/* Procedure to count the total number of races a driver has participated in */
DELIMITER //

DROP PROCEDURE IF EXISTS Get_Driver_Race_Count;

CREATE PROCEDURE Get_Driver_Race_Count(IN p_Driver_ID CHAR(5), OUT raceCount INT)
BEGIN
    SELECT COUNT(*) INTO Race_Count
    FROM Race_Results
    WHERE Driver_ID = p_Driver_ID;
END //

DELIMITER ;
```

```
mysql> CALL Get_Driver_Race_Count('F1001', @Race_Count);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT @Race_Count;
+--------------+
| @Race_Count |
+--------------+
|           68 |
+--------------+
1 row in set (0.00 sec)
```

3. List all race winners in a particular year

```
/* Procedure to get the list of all race winners in a particular year  */
DELIMITER //

DROP PROCEDURE IF EXISTS Get_Race_Winners_By_Year;

CREATE PROCEDURE Get_Race_Winners_By_Year(IN p_Year INT)
BEGIN
    SELECT r.Track, d.Driver_Name
    FROM Race_Results rr
    JOIN Circuits r ON rr.Circuit_ID = r.Circuit_ID
    JOIN Drivers d ON rr.Driver_ID = d.Driver_ID
    WHERE rr.Position = 1 AND rr.Year = p_Year;
END //

DELIMITER ;
```

```
mysql> CALL Get_Race_Winners_By_Year(2023);
+---------------+----------------+
| Track         | Driver_Name    |
+---------------+----------------+
| Bahrain       | Max Verstappen |
| Saudi Arabia  | Sergio Perez   |
| Australia     | Max Verstappen |
| Japan         | Max Verstappen |
| Miami         | Max Verstappen |
| Monaco        | Max Verstappen |
| Canada        | Max Verstappen |
| Spain         | Max Verstappen |
| Austria       | Max Verstappen |
| Great Britain | Max Verstappen |
| Hungary       | Max Verstappen |
| Belgium       | Max Verstappen |
| Netherlands   | Max Verstappen |
| Italy         | Max Verstappen |
| Azerbaijan    | Sergio Perez   |
| Singapore     | Carlos Sainz   |
| United States | Max Verstappen |
| Mexico        | Max Verstappen |
| Brazil        | Max Verstappen |
| Las Vegas     | Max Verstappen |
| Qatar         | Max Verstappen |
| Abu Dhabi     | Max Verstappen |
+---------------+----------------+
22 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

Views:

1. **Top 10 drivers with most points across 3 years**

```sql
/* View to get the top 10 drivers with most points across 3 years */
DROP VIEW IF EXISTS Most_Points_Top10;
CREATE VIEW Most_Points_Top10 AS
SELECT
    DENSE_RANK() OVER (ORDER BY SUM(rr.Points) DESC) AS Position,
    d.Driver_Name,
    SUM(rr.Points) AS Total_Points
FROM Drivers d
JOIN Race_Results rr ON d.Driver_ID = rr.Driver_ID
GROUP BY d.Driver_Name
ORDER BY Total_Points DESC
LIMIT 10;
```

```
mysql> SELECT * FROM Most_Points_Top10;
+----------+-----------------+--------------+
| Position | Driver_Name     | Total_Points |
+----------+-----------------+--------------+
|        1 | Max Verstappen  |         1362 |
|        2 | Charles Leclerc |          803 |
|        3 | Sergio Perez    |          689 |
|        4 | Carlos Sainz    |          668 |
|        5 | Lewis Hamilton  |          657 |
|        6 | George Russell  |          645 |
|        7 | Lando Norris    |          644 |
|        8 | Fernando Alonso |          349 |
|        9 | Oscar Piastri   |          347 |
|       10 | Esteban Ocon    |          168 |
+----------+-----------------+--------------+
10 rows in set (0.00 sec)
```

2. Most race wins across 3 years

```sql
/* View to get the drivers with the most race wins across 3 years  */
DROP VIEW IF EXISTS Most_Wins;
CREATE VIEW Most_Wins AS
SELECT d.Driver_ID, d.Driver_Name, COUNT(rr.Position) AS Total_Wins
FROM Drivers d
JOIN Race_Results rr ON d.Driver_ID = rr.Driver_ID
WHERE rr.Position = 1
GROUP BY d.Driver_ID, d.Driver_Name
ORDER BY Total_Wins DESC;
```

```
mysql> SELECT * FROM Most_Wins;
+-----------+-----------------+------------+
| Driver_ID | Driver_Name     | Total_Wins |
+-----------+-----------------+------------+
| F1001     | Max Verstappen  |         43 |
| F1002     | Charles Leclerc |          6 |
| F1003     | Sergio Perez    |          4 |
| F1005     | Carlos Sainz    |          4 |
| F1007     | Lando Norris    |          4 |
| F1004     | George Russell  |          3 |
| F1006     | Lewis Hamilton  |          2 |
| F1024     | Oscar Piastri   |          2 |
+-----------+-----------------+------------+
8 rows in set (0.00 sec)
```

Triggers:

1. Update total points when new result is added

```sql
/* Trigger to update the total points in the standings after inserting a new result  */
DELIMITER //

DROP TRIGGER IF EXISTS After_Insert_Race_Result;

CREATE TRIGGER After_Insert_Race_Result
AFTER INSERT ON Race_Results
FOR EACH ROW
BEGIN
    UPDATE Driver_Standings
    SET Total_Points = Total_Points + NEW.Points
    WHERE Driver_ID = NEW.Driver_ID;
END //

DELIMITER ;
```

```
mysql> INSERT INTO Race_Results (Circuit_ID, Position, Driver_ID, Team_Name, Sta
rting_Grid, Time_or_Retired, Points, Year) VALUES ('C23', 1, 'F1001', 'Red Bull
Racing', 1, '37:33.6', 26, 2022);
Query OK, 1 row affected (0.04 sec)

mysql> SELECT * FROM Driver_Standings WHERE Final_Position = '1';
+------+----------------+-----------+---------------------------+-------------
+
| Year | Final_Position | Driver_ID | Car                       | Total_Points
|
+------+----------------+-----------+---------------------------+-------------
+
| 2022 |              1 | F1001     | Red Bull Racing RBPT      |          480
|
```

2. Restrict Deletion of race result that has points

```
/* Trigger to restrict deletion of a result with points as it affects the standings  */
DELIMITER //

DROP TRIGGER IF EXISTS Prevent_Delete_Race_Result;

CREATE TRIGGER Prevent_Delete_Race_Result
BEFORE DELETE ON Race_Results
FOR EACH ROW
BEGIN
    IF OLD.Points > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete race result with points';
    END IF;
END //

DELIMITER ;
```

```
mysql> DELETE FROM Race_Results WHERE Points='26';
ERROR 1644 (45000): Cannot delete race result with points
```

Connecting to Python:

I have created 2 files on python, the first is for defined queries and the other one has a menu to insert, update, delete as needed.

Queries:

1. Top 10 driver standings 2024

```python
import mysql.connector

# Database Connection
conn = mysql.connector.connect(
    user="me",
    password="Password123",
    database="F1_21174229"
)

# First Query: Top 10 Driver Standings 2024
cursor1 = conn.cursor()
print("\nQuery 1: Top 10 Drivers Standings 2024:")
cursor1.execute("SELECT * FROM Driver_Standings WHERE Year = 2024  ORDER BY Final_Position ASC LIMIT 10")
for row in cursor1.fetchall():
    print(row)
cursor1.close()
```

```
Query 1: Top 10 Drivers Standings 2024:
(2024, 1, 'F1001', 'Red Bull Racing Honda RBPT', 437)
(2024, 2, 'F1007', 'McLaren Mercedes', 374)
(2024, 3, 'F1002', 'Ferrari', 356)
(2024, 4, 'F1024', 'McLaren Mercedes', 292)
(2024, 5, 'F1005', 'Ferrari', 290)
(2024, 6, 'F1004', 'Mercedes', 245)
(2024, 7, 'F1006', 'Mercedes', 223)
(2024, 8, 'F1003', 'Red Bull Racing Honda RBPT', 152)
(2024, 9, 'F1009', 'Aston Martin Aramco Mercedes', 70)
(2024, 10, 'F1014', 'Alpine Renault', 42)
```

2. Team with most points over the 3 years.

```
# Second Query: Teams with the Most Championship Points
cursor2 = conn.cursor()
print("\nQuery 2: Top 5 Teams with the Most Championship Points:")
cursor2.execute("""
    SELECT Team_Name, SUM(Total_Points) AS Total_Points
    FROM Team_Standings
    GROUP BY Team_Name
    ORDER BY Total_Points DESC
    LIMIT 5
""")
for row in cursor2.fetchall():
    team_Name, total_Points = row
    print((team_Name, int(total_Points)))
cursor2.close()
```

```
Query 2: Top 5 Teams with the Most Championship Points:
('Red Bull Racing', 2208)
('Ferrari', 1612)
('Mercedes', 1392)
('McLaren', 1127)
('Aston Martin', 429)
```

3. Most expensive tickets (with type and location)

```
# Third Query: Most Expensive Tickets
cursor3 = conn.cursor()
print("\nQuery 3: Top 5 Most Expensive Tickets:")
cursor3.execute("""
    SELECT Ticketing.Ticket_ID, Circuits.Track, Tickets.Type, Ticketing.Price
    FROM Ticketing
    JOIN Tickets ON Ticketing.Ticket_ID = Tickets.Ticket_ID
    JOIN Circuits ON Ticketing.Circuit_ID = Circuits.Circuit_ID
    ORDER BY Ticketing.Price DESC
    LIMIT 5
""")
for row in cursor3.fetchall():
    ticket_id, track, ticket_type, price = row
    print((ticket_id, track, ticket_type, float(price)))
print("")
cursor3.close()


# Close connection
conn.close()
```

```
Query 3: Top 5 Most Expensive Tickets:
(8, 'Abu Dhabi', 'Paddock Club', 7700.0)
(8, 'Monaco', 'Paddock Club', 7500.0)
(8, 'Miami', 'Paddock Club', 6500.0)
(8, 'Great Britain', 'Paddock Club', 5300.0)
(8, 'United States', 'Paddock Club', 5300.0)
```

Interaction Menu

1. Functions to insert, update and delete data on the database.

```
import mysql.connector

conn = mysql.connector.connect(
    user="me",
    password="Password123",
    database="F1_21174229"
)
cursor = conn.cursor()

# Insert New Team
def insert_team():
    team_name = input("Enter team name: ")
    cursor.execute("SELECT * FROM Teams WHERE Team_Name = %s", (team_name,))
    if cursor.fetchone() is not None:
        print("Error: A team with this name already exists.")
    else:
        base = input("Enter team base: ")
        cursor.execute("INSERT INTO Teams (Team_Name, Base) VALUES (%s, %s)", (team_name, base))
        conn.commit()
        print("Team inserted successfully.")


# Insert New Driver
def insert_driver():
    driver_id = input("Enter driver ID (5 characters): ")
    cursor.execute("SELECT * FROM Drivers WHERE Driver_ID = %s", (driver_id,))
    if cursor.fetchone() is not None:
        print("Error: A driver with this ID already exists.")
    else:
        driver_number = int(input("Enter driver number: "))
        driver_name = input("Enter driver name: ")
        team_name = input("Enter team name: ")
        dob = input("Enter date of birth (YYYY-MM-DD): ")
        nationality = input("Enter nationality: ")
        cursor.execute("INSERT INTO Drivers VALUES (%s, %s, %s, %s, %s, %s)",
                       (driver_id, driver_number, driver_name, team_name, dob, nationality))
        conn.commit()
        print("Driver inserted successfully.")

# Insert New Race Result
def insert_race_result():
    year = int(input("Enter year: "))
    circuit_id = input("Enter circuit ID (3 characters): ")
    driver_id = input("Enter driver ID: ")

    cursor.execute("SELECT * FROM Race_Results WHERE Year = %s AND Circuit_ID = %s AND Driver_ID = %s",
                   (year, circuit_id, driver_id))
    if cursor.fetchone() is not None:
        print("Error: A race result for this driver in this circuit and year already exists.")
    else:
        position = int(input("Enter position: "))
        team_name = input("Enter team name: ")
        starting_grid = int(input("Enter starting grid position: "))
        time_or_retired = input("Enter race time or 'Retired': ")
        points = int(input("Enter points: "))
        cursor.execute("INSERT INTO Race_Results VALUES (%s, %s, %s, %s, %s, %s, %s, %s)",
                       (year, circuit_id, position, driver_id, team_name, starting_grid, time_or_retired, points))
        conn.commit()
        print("Race result inserted successfully.")
```

```python
# Update Team Base
def update_team_base():
    team_name = input("Enter team name to update: ")
    cursor.execute("SELECT * FROM Teams WHERE Team_Name = %s", (team_name,))
    if cursor.fetchone() is None:
        print("Error: Team not found.")
    else:
        new_base = input("Enter new base: ")
        cursor.execute("UPDATE Teams SET Base = %s WHERE Team_Name = %s", (new_base, team_name))
        conn.commit()
        print("Team updated successfully.")

# Update Driver's Team
def update_drivers_team():
    driver_id = input("Enter driver ID to update: ")
    cursor.execute("SELECT * FROM Drivers WHERE Driver_ID = %s", (driver_id,))
    if cursor.fetchone() is None:
        print("Error: Driver not found.")
    else:
        new_team = input("Enter new team name: ")
        cursor.execute("UPDATE Drivers SET Team_Name = %s WHERE Driver_ID = %s", (new_team, driver_id))
        conn.commit()
        print("Driver updated successfully.")


# Delete team
def delete_team():
    team_name = input("Enter team name to delete: ")
    cursor.execute("SELECT * FROM Teams WHERE Team_Name = %s", (team_name,))
    if cursor.fetchone() is None:
        print("Error: Team not found.")
    else:
        cursor.execute("DELETE FROM Teams WHERE Team_Name = %s", (team_name,))
        conn.commit()
        print("Team deleted successfully.")

# Delete driver
def delete_driver():
    driver_id = input("Enter driver ID to delete: ")
    cursor.execute("SELECT * FROM Drivers WHERE Driver_ID = %s", (driver_id,))
    if cursor.fetchone() is None:
        print("Error: Driver not found.")
    else:
        cursor.execute("DELETE FROM Drivers WHERE Driver_ID = %s", (driver_id,))
        conn.commit()
        print("Driver deleted successfully.")
```

2. Menu and Options

```
# Menu
def menu():
    choice = "1"
    while choice != "10":
        print("\nFORMULA 1 DB MANAGEMENT MENU:")
        print("1. Insert Team")
        print("2. Insert Driver")
        print("3. Insert Race Result")
        print("4. Update Team")
        print("5. Update Driver")
        print("6. Delete Team")
        print("7. Delete Driver")
        print("8. Exit")

        choice = input("Enter your choice: ")


        if choice == "1":
            insert_team()
        elif choice == "2":
            insert_driver()
        elif choice == "3":
            insert_race_result()
        elif choice == "4":
            update_team_base()
        elif choice == "5":
            update_drivers_team()
        elif choice == "6":
            delete_team()
        elif choice == "7":
            delete_driver()
        elif choice == "8":
            print("Menu closed.")
        else:
            print("Invalid choice. Please try again.")


menu()
cursor.close()
conn.close()
```

3. Terminal Interaction Samples:

```
FORMULA 1 DB MANAGEMENT MENU:
1. Insert Team
2. Insert Driver
3. Insert Race Result
4. Update Team
5. Update Driver
6. Delete Team
7. Delete Driver
8. Exit
Enter your choice: 1
Enter team name: CUD
Enter team base: Dubai, UAE
Team inserted successfully.
```

```
FORMULA 1 DB MANAGEMENT MENU:
1. Insert Team
2. Insert Driver
3. Insert Race Result
4. Update Team
5. Update Driver
6. Delete Team
7. Delete Driver
8. Exit
Enter your choice: 4
Enter team name to update: CUD
Enter new base: Dubai, United Arab Emirates
Team updated successfully.
```

```
FORMULA 1 DB MANAGEMENT MENU:
1. Insert Team
2. Insert Driver
3. Insert Race Result
4. Update Team
5. Update Driver
6. Delete Team
7. Delete Driver
8. Exit
Enter your choice: 6
Enter team name to delete: Curtin
Error: Team not found.
```

Discussion:

The assignment went really well, it gave me a better understanding on how to approach and design real life databases. Even though, most parts were easy, I had to invest a lot of time in some confusing parts. I spent a lot of time figuring out the best possible entities and relationships. After that was done, the following steps were easy.

After that, I started with the implementation of the database, which was also fine as I completely followed my relational schema and data description. I had also spent a lot of time inputting data onto the database. Creating queries and advanced concepts were clear and easy to implement. Python was another part that was confusing and time-consuming. I had issues related to ssl, which was a technical issue with my system. Fixing the ssl was a lengthy process and it took time, because the ubuntu system crashed twice while fixing the ssl and I had to reinstall.

Some things I could have improved are, adding more attributes to the tables so that users can find anything they want related to F1. I also think that I should have looked for alternative inserting options such as inserting directly from an excel file.